

네트워크 게임프로그래밍 텀프로젝트

8팀 추진계획서

2020180029	이준호
2020112036	정현도
2020182045	황수지

목차

- 01. 기획
 - 1) 개요
 - 2) 기존 게임 요소 및 시스템
 - 3) 프로젝트 변경 및 추가 요소
 - 4) 개발 환경
- 02. High-level 디자인
 - 1) 서버 - 클라이언트 구조
 - 2) 서버 구조
 - 3) 클라이언트 구조
- 03. Low-level 디자인
 - 1) 서버 - 클라이언트
 - 2) 서버
 - 3) 클라이언트
- 04. 역할 분담
- 05. 개발 일정

01. 기획

1) 개요

이름	Dark Knight
장르	플랫폼 게임 (횡스크롤 액션 게임)
내용	플레이어는 다양한 무기를 사용해 몬스터를 처치하고, 장애물을 파괴하며 점수를 획득해 스테이지를 클리어하는 것이 목표.
제작 교과목	2023 윈도우 프로그래밍 (정현도, 이준호 제작)

2) 기존 게임 요소 및 시스템

- 플레이어

① 조작 방법

좌우 이동	방향키 ←, →
포탈 출입	방향키 ↑
점프	SPACE
무기 변경	TAP
공격	A

② 무기종류

검		근거리 공격 + 트랩 제거 기능
활		원거리 공격
폭탄		원거리 공격 + 트랩 제거 기능

③ 아이템

포션		일정 체력 회복
스타		트랩 제거 시 일정 확률 생성. 공격력 향상

- 장애물

트랩		닿으면 데미지 입는다. 폭탄 공격 시 제거됨.
----	---	------------------------------

- 몬스터 종류

근거리 몬스터		좌우로 움직이고, 닿으면 데미지를 입음
원거리 몬스터1		가만히 서 있는 상태에서 원거리 공격을 한다. 공격에 맞으면 데미지 입음. 공격을 플레이어의 공격으로 상쇄 시킬 수 있음.
원거리 몬스터2		가만히 서 있는 상태에서 플레이어의 위치로 공격을 떨어뜨림. 공격을 플레이어의 공격으로 상쇄 시킬 수 있음.
보스 몬스터	- 플레이어가 일정범위 안에 다가오면 자동적으로 공격함. - 일정 시간마다 3가지의 공격 패턴 중 랜덤하게 공격함. - 보스가 울음소리를 낼 때는 공격이 통하지 않음. <공격 패턴>    	① 기본 : 플레이어가 일정범위 안에 다가오면 자동적으로 공격함. ② 랜덤1 : 플레이어의 방향으로 3개의 공격을 날린다. 플레이어는 이 공격을 피하거나 검은 베어 회피할 수 있다. ③ 랜덤2 : 보스가 자신의 부하를 불러 같이 싸우도록한다. ④ 랜덤3 : 보스가 순간적으로 사라지며 일정시간 이후 플레이어 주변에 나타나 기본공격을 가한다.

- 스테이지 구성

스테이지1	 간단한 지형으로 장애물을 부숴가며 근거리 몬스터들과 원거리 몬스터들을 처치해야함.
스테이지2	 다양한 패턴의 발판 존재하는 지형으로 낙사할 수 있음. 원거리 몬스터도 추가되어 난이도가 올라감.
보스 스테이지	 맵은 기본적으로 간단하며 보스와 싸우는 장소에 공격을 피할 수 있도록 순간이동하는 포탈이 있는 발판이 있음. 보스를 물리치면 게임 클리어.

3) 프로젝트 변경 및 추가 요소

- 싱글 -> 2인 멀티 플레이
- 무기 선택(검/활/폭탄) -> 역할 선택 (검사/궁수)

검사		근거리 공격 담당
궁수		원거리 공격 담당

- 로그인, 로비, 방 씬

로그인, 회원가입

아이디

비밀번호

로그인

회원가입

로비
(방 생성, 입장 기능)

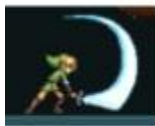
바로입장

방 만들기

번호	방 제목	인원	
1	초보자 환영	(1/2)	입장
2	즐거운 게임 해요	(1/2)	입장
3	고수만 들어와	(2/2)	입장
4	궁수 있어요	(1/2)	입장

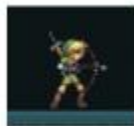
방
(역할 선택 및 게임 준비 기능)

Player1



준비중

Player2



준비완료

게임 시작

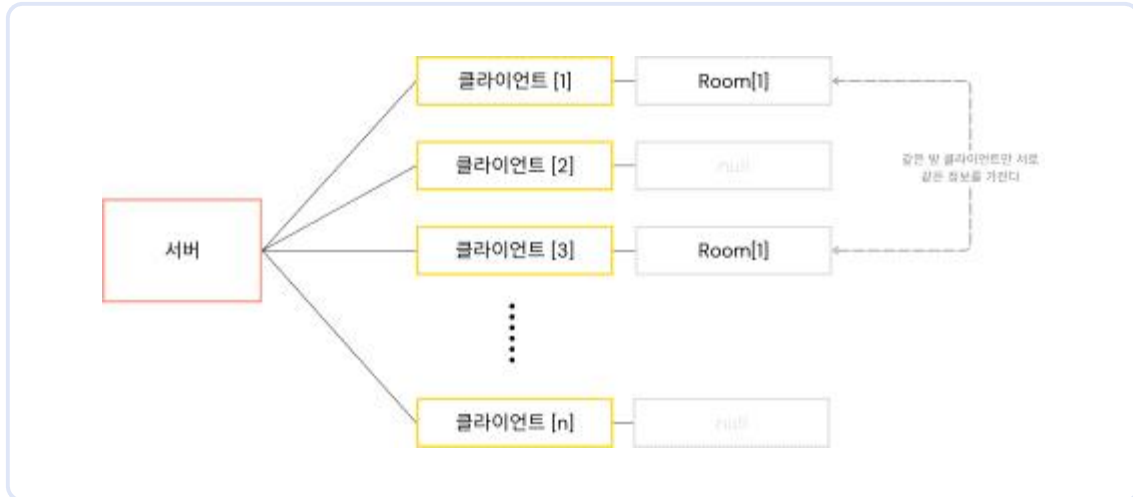
4) 개발 환경

- 개발 도구: Visual Studio 2022
- 운영 체제: Windows 11
- API: Windows API (WinAPI)
- 네트워크 프로토콜: TCP/IP
- 네트워크 라이브러리: WinSock
- 개발 언어: C/C++
- 협업 도구: GitHub

02. High-level 디자인

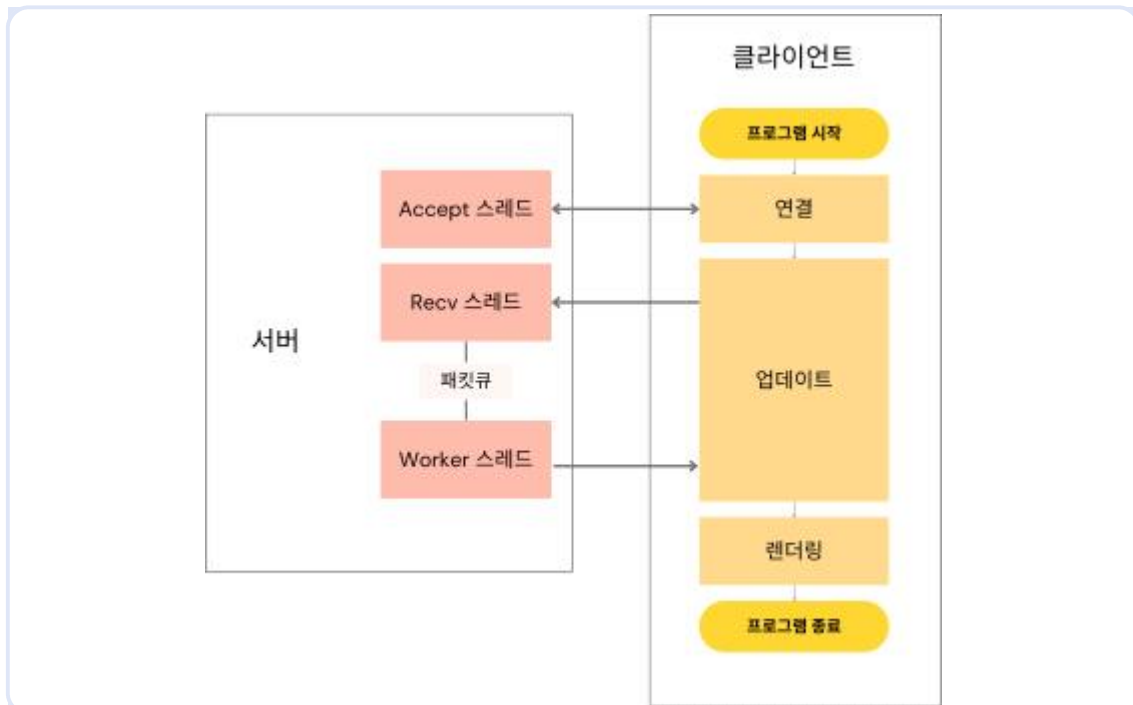
1) 서버 - 클라이언트 구조

● 다중 클라이언트 처리



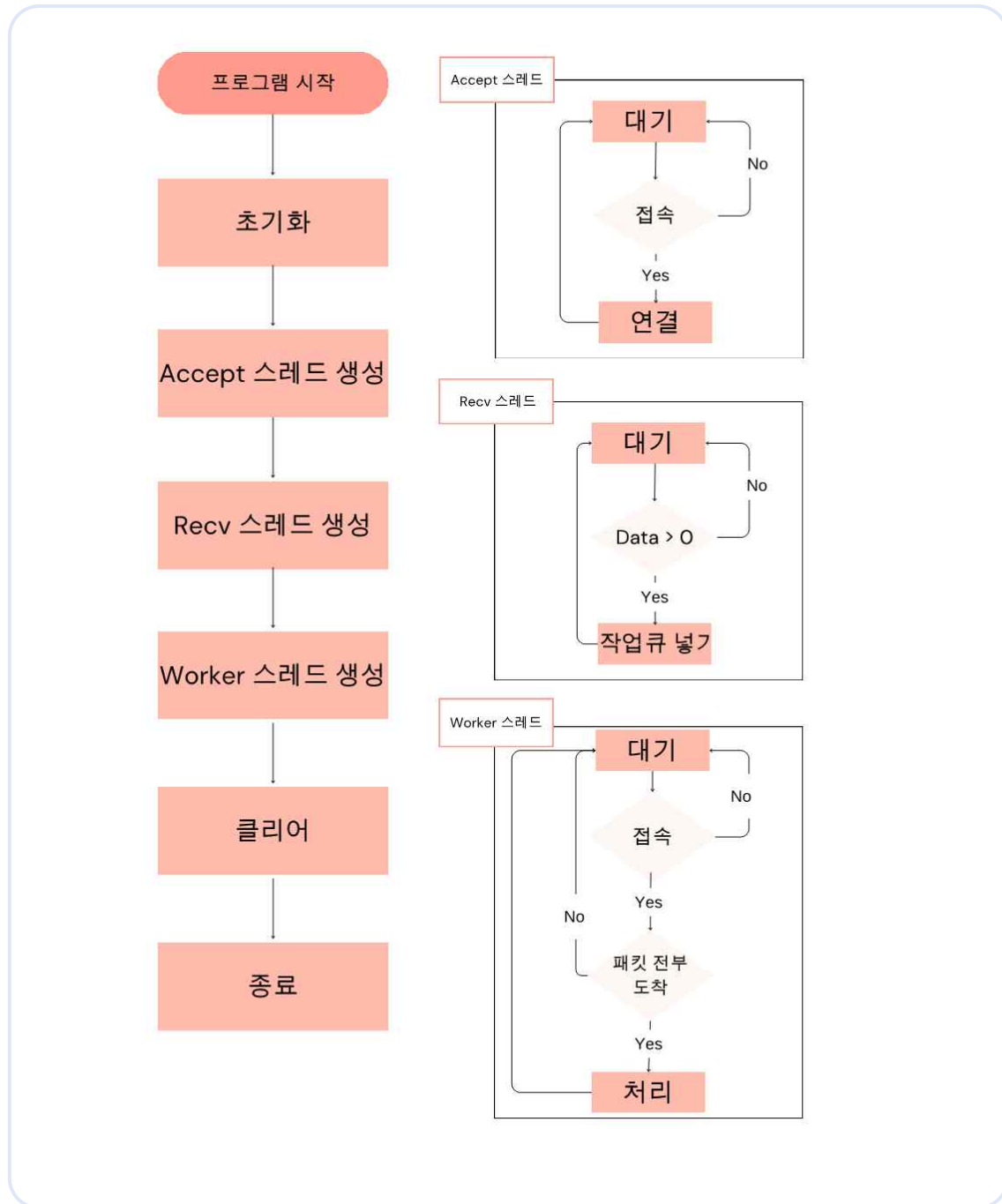
- 게임 정보는 모두 서버가 가지고 있다.
- 같은 방 번호를 가진 클라이언트는 서로 같은 게임 정보를 서버에게 받아 동기화한다.

● 개별 클라이언트에 따른 구조

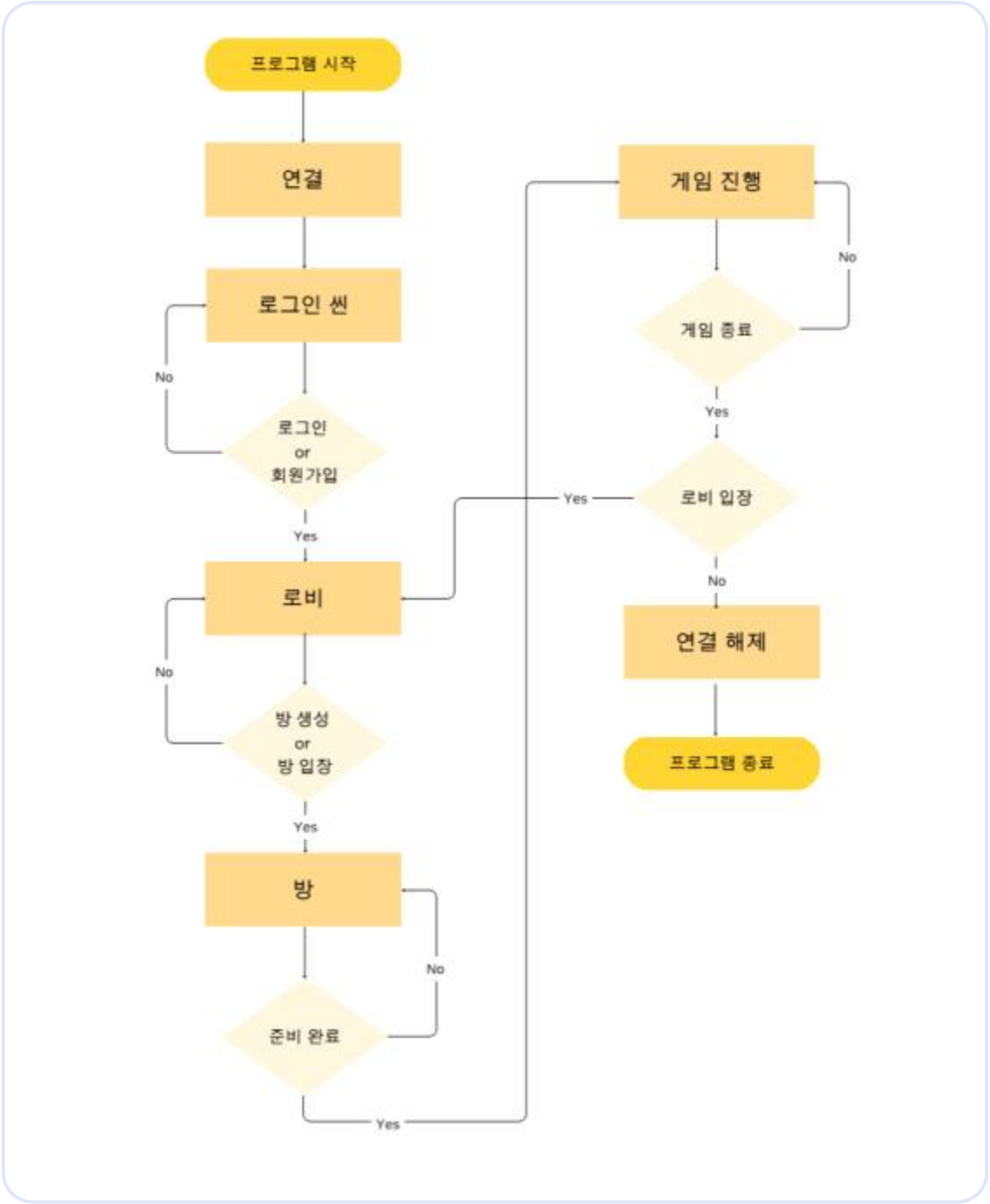


- 서버는 Accept스레드에서 접속하는 클라이언트의 연결을 처리한다.
- 서버는 Recv스레드에서 클라이언트의 이벤트에 따라 전송된 패킷을 받아 패킷큐에 저장한다.
- 패킷큐에 저장된 패킷타입에 따라 작업(이동, 충돌처리...)을 하고 결과(이동 좌표, hp...)를 클라이언트에게 전송하고, 클라이언트는 이에 따라 게임정보를 업데이트한다.

2) 서버 구조



3) 클라이언트 구조



03. Low-level 디자인

1) 서버 - 클라이언트

① 구조체 타입

Room		
타입	멤버변수명	역할
int	id	식별 번호
char	state	진행 상태 (대기/준비/진행/종료)
char	stage	진행 중 스테이지 단계
int	total_time	클리어 시간

PlayerInfo		
타입	멤버변수명	역할
int	id	식별 번호
int	x, y	위치
short	hp	체력
short	damage	공격력
char	type	검사/궁수 역할
char	state	Idle/점프/공격

MonsterInfo		
타입	멤버변수명	역할
int	id	식별 번호
char	type	몬스터 종류
int	x, y	위치
short	hp	체력
short	damage	공격력
char	direct	공격/이동 방향
char	attack_type	공격 패턴 (보스의 경우)

TileInfo		
타입	멤버변수명	역할
short	type	맵 이미지 종류
int	x, y	위치

ItemInfo		
타입	멤버변수명	역할
char	type	종류 (포션/스타/장애물)
int	x, y	위치
short	amount	아이템의 능력치

② 패킷 타입

Packet		
타입	멤버변수명	역할
char	type	패킷 종류
int	length	패킷 사이즈
vector<char>	data	패킷에 전송할 정보들

PacketType		
열거자	설명	전송자
InputKey	Input정보 제공	Client
LoginRequest	로그인 요청	Client
LogoutRequest	로그아웃 요청	Client
RegisterRequest	회원가입 요청	Client
LoginSuccess	로그인 정보 일치	Server
LoginFailure	로그인 정보 불일치, 로그인 재요청	Server
RegisterSuccess	회원가입 성공	Server
RegisterFailure	회원가입 정보 중복으로 실패	Server
RoomCreateRequest	방 생성 요청	Client
RoomJoinRequest	방 입장 요청	Client
RoomLeaveRequest	방 나가기 요청	Client
RoomInfoUpdate	방 입장 정보 제공	Server
LobbyInfoUpdate	로비에 있는 플레이어에게 방 목록 제공	Server
GameStartRequest	게임 준비 완료 시 게임 시작 요청	Client
GameEnd	방 게임 종료 처리 요청	Client
PlayerMove	플레이어 이동 요청	Client
PlayerAttack	플레이어 공격 요청	Client
GameStartResponse	방 정보 상태 업데이트 및 맵 정보 제공	Server
GameEndNotification	게임 종료 정보 처리 및 제공	Server
PlayerInfoUpdate	플레이어 이동 및 상태 업데이트	Server
MonsterInfoUpdate	몬스터 이동 및 상태 업데이트	Server

2) 서버

① 변수

타입	멤버변수명	역할
map<int, Client>	clients	접속한 클라이언트들을 관리
vector<Room>	rooms	모든 룸 관리
queue<Packet>	packet_queue	recv스레드에서 저장되고 worker스레드에서 처리할 데이터

② 구조체

Client		
타입	멤버변수명	역할
PlayerInfo	player	플레이어 정보저장
Room	my_room	클라이언트가 속한 방 정보

ScenelInfo		
타입	멤버변수명	역할
int	id	식별 번호
char	stage	진행 중 스테이지 단계
int	total_time	클리어 시간
array<vector<object>, 4>	objects	현재 맵의 오브젝트

③ 함수

Thread			
타입	함수명	인자	기능
void	AcceptThread		클라이언트 연결 처리하는 스레드
void	RecvThread		recv 처리를 해서 패킷큐에 넣는 스레드
void	WorkerThread		패킷 큐에 정보가 있으면 패킷을 처리하는 스레드

Network			
타입	함수명	인자	기능
void	ProcessPackets	Packet& Packet	패킷을 꺼내 타입 별로 처리하는 함수
void	SendLoginResponse	int PlayerID	로그인 요청에 결과 보내는 함수
void	SendRegisterResponse	int PlayerID	회원가입 요청에 결과 보내는 함수
void	SendPlayerInfo	int PlayerID	플레이어 정보를 전송하는 함수
void	SendLobbyInfo	int PlayerID	로비 정보를 전송하는 함수
void	SendRoomCreateResponse	int PlayerID	방 생성 요청에 결과 보내는 함수
void	SendRoomJoinResponse	int PlayerID	방 입장 요청에 결과 보내는 함수
void	SendRoomInfo	int PlayerID	방 정보를 전송하는 함수
void	SendGameStartResponse	int PlayerID	게임 시작 요청에 결과 보내는 함수
void	SendEndNotification	int PlayerID	게임 종료 요청에 결과 보내는 함수
void	SendMonsterInfo	int PlayerID	몬스터 정보를 전송하는 함수

Colision			
타입	함수명	인자	기능
void	ProcessCollision	int PlayerID	충돌 처리하고 충돌 결과 리턴
bool	IsCollisionMonster	int PlayerID	
bool	IsCollisionTile	int PlayerID	
bool	IsCollisionItem	int PlayerID	

3) 클라이언트

Scene		
타입	멤버변수명	역할
vector<RECT>	button	버튼
int	num	식별번호

LoginScene : public Scene		
타입	멤버변수명	역할
char*	id	서버로 보낼 id
char*	passwd	서버로 보낼 passwd

LobbyScene : public Scene		
타입	멤버변수명	역할
vector<Room>	rooms	서버에서 로비에 있는 방들 정보 저장

RoomScene : public Scene		
타입	멤버변수명	역할
PlayerInfo	player[2]	플레이어 저장
Room	my_room	현재 방 정보 저장

GameScene : public Scene		
타입	멤버변수명	역할
array<vector<Object>, 4>	objects	타일 맵, 적 객체, 아이템, 트랩
PlayerInfo	player[2]	플레이어 저장

ClientNetwork			
타입	멤버변수명	역할	
SOCKET	socket	통신을 위한 클라이언트의 소켓	
타입	함수명	인자	기능
void	SendLoginRequest	char* id, char* password	로그인 요청
void	SendRegisterRequest	char* id, char* password	회원가입 요청
void	SendLogoutRequest		로그아웃 요청
void	SetUserData		유저데이터를 받음
void	RecvRobbyData		로비 정보를 받아서 업데이트함
void	ChangeScene		씬 변경
void	SendJoinRoomRequest	int RoomID	방 입장을 요청함
void	SendCreateRoomRequest		방 생성을 요청함
void	RecvRoomData		방 정보를 받음, 초기 플레이어
void	SendChangeCharacter		캐릭터 정보 변경을 요청
void	SendReadyState		플레이어 준비 정보를 보냄
void	SendExitRoom		플레이어 방에서 나감
void	RecvRoomData		방 정보, 플레이어 정보를 받고 업데이트함
void	RecvGameSceneData		게임씬 정보를 받음
void	SendPlayerData		플레이어 정보, 키 입력을 보냄
void	RecvPlayerData		시뮬레이션 된 캐릭터 정보를 받음
void	SetPlayerData		받은 캐릭터 정보를 적용함
void	RecvGameSceneData		게임씬 정보를 받음
void	SetGameSceneData		받은 씬 정보를 적용시킴
void	SendQuitScene		퇴장 정보를 보냄
void	SendInputKey	UINT message, WPARAM wParam, LPARAM lParam	입력 정보를 보냄

04. 역할 분담

역할	할일	정현도	이준호	황수지
서버	네트워크 통신 함수 구현			
	오브젝트들 클래스 구현			
	Accept 스레드 구현			
	Recv 스레드 구현			
	Work 스레드 구현+			
	오브젝트 충돌 처리 함수 구현			
클라이언트	네트워크 통신 함수 구현			
	Scene 코드 개선			
	Login 로직 구현 및 UI 요소 추가			
	Lobby 로직 구현 및 UI 요소 추가			
	Game 코드 개선			
	Result 로직 구현 및 UI 요소 추가			
	Room 로직 구현 및 UI 요소 추가			
	Input 관리 코드 개선			
	Objects 관리 코드 개선			
기타	네트워크 통신 프로토콜 설계			
	문서작성/편집			

05. 개발 일정 (11/3 ~ 12/5)

1) 정현도

주	일	월	화	수	목	금	토
1	3	4	5	6	7	8	9
	Scene 코드 개선				네트워크 통신 함수 구현		네트워크 통신 함수 구현
2	10	11	12	13	14	15	16
		Login 로직 구현		Login UI 요소 추가		Lobby 로직 구현	
3	17	18	19	20	21	22	23
	Lobby UI 요소 추가		Game 씬코드		Accept 스레드 구현		
4	24	25	26	27	28	29	30
	오브젝트 충돌 처리 함수 구현						
5	1	2	3	4	최종발표		
	디버깅						

2) 이준호

주	일	월	화	수	목	금	토
	3	4	5	6	7	8	9
1	Game 코드 개선						Result로직 구현 Result UI요소추가
	10	11	12	13	14	15	16
2	Room 로직 구현 및 UI요소 추가						
	17	18	19	20	21	22	23
3	Input관리 코드개선		Input관리 코드개선		Objects관리 리코드개선		
	24	25	26	27	28	29	30
4	Objects관리코드개선				Recv 스레드 구현		
	1	2	3	4			
5	디버깅 및 테스트트				최종발표		

3) 황수지

주	일	월	화	수	목	금	토
1	3	4	5	6	7	8	9
	네트워크 통신 프로토콜 설계						
2	10	11	12	13	14	15	16
	오브젝트들 클래스 구현 (Client, Monster)			오브젝트들 클래스 구현 (Room,Tile)			
3	17	18	19	20	21	22	23
	Work 스레드 구현				Work 스레드 구현		
4	24	25	26	27	28	29	30
	Work 스레드 구현						
5	1	2	3	4	최종발표		
	디버깅						